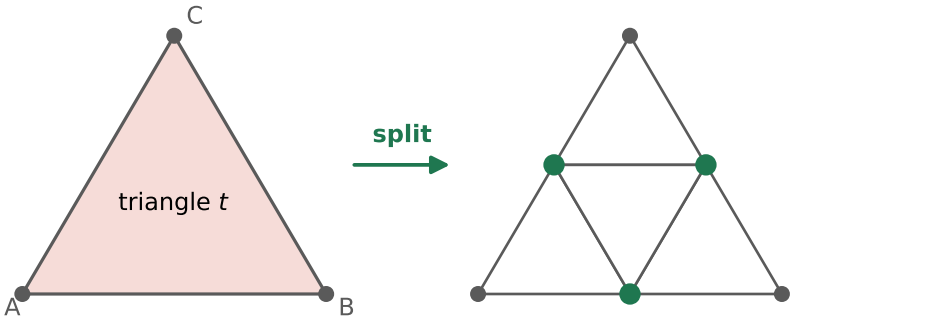
(a) priced split: does this triangle deserve 30 more bytes?

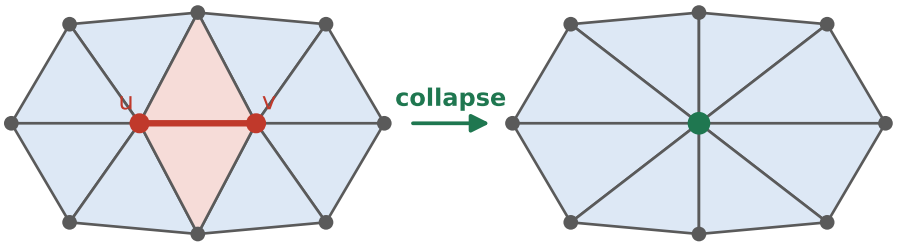


3 new vertices (edge midpoints), 1 parent -> 4 children

Stage 1, every 500 it until 10k: split t iff $\Delta D_t/\Delta R_t > \lambda$

ΔD_t: in each training view, sum the L1 error of the pixels
whose front triangle is t, divide by image Mpx, average over views
e.g. 40 px x 0.05 = 2.0 ; / 1.6 Mpx = 1.25 ; visible in 30 % of views -> 0.375
ΔR_t = 3 new vertices x B_v = 3 x 10 B = 30 B
ratio = 0.375 / 30 = 0.0125
λ = 6.2e-3 : 0.0125 > λ -> split
λ = 1.85e-2: 0.0125 < λ -> keep (growth stops here earlier)

(b) lambda-prune: is this vertex worth the 10 bytes it costs?



10 faces in the two rings
(red = the 2 faces on edge uv)

merged vertex (green) at the midpoint

8 faces: -1 vertex, -2 faces

Stage 2, at 12k / 14k / 16k / 18k: collapse (u, v) iff $\Delta D_{uv}/\Delta R_{uv} < \lambda$

$\Delta D_{uv} = D_u + D_v - D_{uv}$: residual of all faces touching u or v
(same per-view, per-Mpx units as the split), e.g. 10 faces x 0.004 = 0.04
ΔR_{uv} = bytes saved = $(B_v - 2B_f)$ + faces removed x B_f
interior edge: (10 - 4) + 2 x 2 = 10 B (boundary 8 B, non-manifold 12 B)
score = 0.04 / 10 = 0.004
λ = 6.2e-3 : 0.004 < λ -> collapse
λ = 1.85e-3: 0.004 > λ -> keep
per round: no vertex in two collapses, cheapest first, <= 15 % of V,
reject if a surviving face would flip

Numbers are illustrative. One lambda drives both rules; B_v = 10 B already includes ~4 B of topology, B_f = 2 B.